

Photon Splatting for Participating Media

Antoine Boudet^{1,2}, Paul Pitot², David Pratumarty² and Mathias Paulin¹

¹IRIT-UPS-CNRS, 118 Route de Narbonne, 31062 Toulouse, France

²OKTAL SE, 2 Rue Boudeville, 31000 Toulouse, France

Abstract

Since the beginning of image synthesis, much research have been done on global illumination simulation. However, simulation in participating media is still an open problem as far as computing time is concerned. Recently some methods, like Photon Mapping, proposed an optimization of the resolution of global illumination in participating media. Nevertheless, the computing costs of these methods remain very expensive.

In this paper, we present a method which takes advantage of density estimation to efficiently reconstruct volume irradiance from the photon map. Our idea is to separate the computation of emission, absorption and out-scattering from the computation of in-scattering. Then we use a dual approach of density estimation to optimize this last part as it is the most computational expensive. Our method extends Photon Splatting, which optimizes the computation time of Photon Mapping for surface rendering, to participating media, and then considerably reduce participating media rendering times. Even though our method is faster than Photon Mapping for equal quality, we also propose a GPU based optimization of our algorithm.

Keywords: Photon Splatting, participating media, global illumination, density estimation

1 Introduction

Nowadays, a large number of applications need to take participating media into account. This kind of rendering can be used in very different fields, from the military field - remote sensing simulation, battlefield smoke plumes - to the industrial field - design of efficient headlamps for foggy driving - and much more.

To answer these needs, numerous methods have been proposed for participating media rendering. But these methods suffer from two main problems. Firstly, they often make drastic hypothesis on media complexity, i.e. homogeneity and isotropy. Secondly, the computing time needed to solve the global illumination is very expensive. In fact, the most expensive part of the calculation is solving multiple scattering effect. The absorption, emission and single scattering are quite simple to solve : we use a ray marching like method to compute these phenomena contribution which gives good results.

In this paper, our goal is to considerably reduce the computing cost for multiple scattering. We present a method which optimizes the computing of these effects. Our method is based on Photon Mapping [Jensen 2001] in order to construct a volume photon map. Then we use density estimation techniques to speed up the evaluation of the contribution of in-scattering from the photon map, the other phenomena being computed using classical ray marching. We adapted the Photon Splatting method [Stuerzlinger and Bastos

1997], which was designed for surface rendering (i.e. 2D density estimation), in order to do volumetric density estimation needed for participating media rendering.

Our paper is organised as follows. Firstly, in section 2, we present the fundamentals of participating media theory. Then, we briefly present the previous methods for participating media rendering, the Photon Splatting method [Stuerzlinger and Bastos 1997] and the Photon Mapping method (section 3). Section 4 presents our main contribution. In section 4.1, we present the iterative formulation of the volume radiative transfer equation we use. This expression permits to treat separately emission, absorption and out-scattering on one hand (section 4.2) and in-scattering on the other hand (section 4.3) using distinct suited methods. Then we present our results in section 4.4, and finally we propose a GPU based optimization of our algorithm in section 5.

2 Participating Media Theory

2.1 Overview

Light travels through vacuum without energy modification. On the contrary, a participating media affects light propagation. Three phenomena can be considered:

- Emission (figure 1(a)) : Light energy that can be emitted by the media, particularly in the Infrared (IR) domain, or for high temperature media in visible spectrum (like fire).
- Absorption (figure 1(b)) : Light energy that can be absorbed by the media (like dense smoke).
- Scattering : Light energy that can be lost or gathered on the path of a ray. For convenience, this phenomena can be splitted in two cases. The first is the light energy lost on the path of a ray in the media, which is called out-scattering (figure 1(c)). The second is the light energy that has been scattered by the media in the direction of observation. This is called in-scattering (figure 1(d)).

2.2 Volume Radiative Transfer Equation

The radiative transfer in participating media can be formalized by the volume transfer equation. This is the integral expression of the radiative transfer equation in participating media [Chandrasekar 1950]. Considering point x in the media as in figure 2, the radiance in direction ω_0 , $L(x, \omega_0)$ (abbreviated $L(x)$ for simpler notations), is given by:

$$\begin{aligned} L(x) = & e^{-\tau(x_0, x)} L(x_0) \\ & + \int_{x_0}^x e^{-\tau(u, x)} \sigma_a(u) L_e(u) du \\ & + \int_{x_0}^x e^{-\tau(u, x)} \frac{\sigma_s(u)}{4\pi} L_\omega(u) d\omega \end{aligned} \quad (1)$$

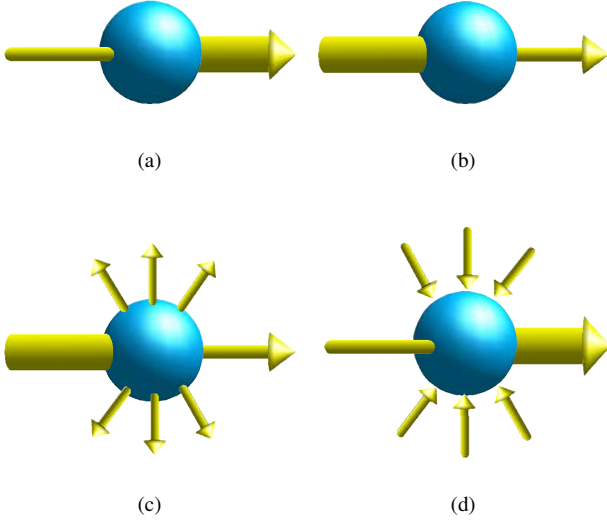


Figure 1: Phenomena affecting light propagation. (a) Emission. (b) Absorption. (c) Out-scattering. (d) In-scattering.

where

$$L_\omega(x) = \int_{\Omega=4\pi} p(x, \omega_0, \omega_i) L(x, \omega_i) d\omega_i \quad (2)$$

is the in-scattering radiance at point x , σ_a is the absorption coefficient (equal to the emission coefficient at thermodynamic equilibrium), σ_s is the scattering coefficient, L_e is the self emitted radiance at x , $p(x, \omega_0, \omega_i)$ is the media phase function, and $L(x, \omega_i)$ the incoming radiance from direction ω_i . $\tau(a, b) = \int_a^b \sigma_t(u) du$ is the optical thickness on the path $[a, b]$ with $\sigma_t = \sigma_a + \sigma_s$ the extinction coefficient. Detailed demonstration of this equation can be found in [Siegel and Howell 1992].

The first term of equation 1 corresponds to the loss of energy due to absorption and out-scattering on the path $[x_0, x]$. The second and third terms correspond to the gain of energy due respectively to self emission and in-scattering on the same path.

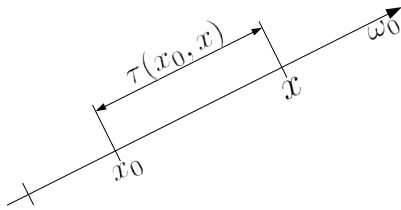


Figure 2: Geometry associated with the volume transfer equation.

3 Related Works

In this section, we present a quick overview of the participating media rendering methods. Then we present the Photon Splatting method which is the base of our work. Finally, we present and discuss the Photon Mapping method applied to participating media.

3.1 Participating media simulation

Over the last twenty years, a lot of methods have been developed to simulate participating media. According to [Perez et al. 1997], these methods can be classified into two main categories : deterministic and stochastic methods.

The principle of deterministic methods is to use a spatial or angular discretisation of the radiative transfer equation. The resulting linear equations system is then solved using finite element (zonal method [Rushmeier and Torrance 1987]) or discrete ordinate methods [Max 1994]. But these methods, which work well for simple scenes, are quite unapplicable in complex environments due to the huge number of equations that have to be solved yielding very expensive computing costs. In order to have a good overview of deterministic methods, you can refer to [Perez et al. 1997].

The principle of stochastic methods is to solve the transport equation using random sampling of paths between interaction points. These methods use ray tracing techniques in order to construct the sampled paths and use the Monte Carlo method for importance sampling. Then computing time can be spent in more important calculations. Monte Carlo raytracing methods, whose survey can be found in [Lszl 1999], have been extended to handle participating media. Most of these methods work in image space and then are view dependent [Blasi et al. 1993; Blasi et al. 1994; Pattanaik and Mudur 1993; Lafortune and Willems 1996; Veach and Guibas 1997]. All these methods give accurate results but are computationally expensive. Today, the most interesting methods are derived from Photon Mapping [Jensen 2001]. We make a discussion of this method in section 3.3 in order to explain the drawback we had to solve.

Some interesting work has been done on real time rendering of participating media [Nishita et al. 1996; Harris and Lastra 2001] but, in order to achieve the real time rendering, they make constraining approximations and do not take all multiple scattering effects into account.

3.2 Photon Splatting

Photon Splatting [Stuerzlinger and Bastos 1997; Lavignotte and Paulin 2003] purpose is to render scenes with glossy global illumination. It is based on both photon tracing and density estimation. This is a two pass method. Firstly, photons are traced toward the scene and stored onto the surfaces they hit. In the second pass, the stored photons are used to estimate the radiance of the surfaces. The outgoing radiance in direction ω_0 is given using estimation of photon density around the surface point x . In order to achieve a smooth reconstruction of the radiance, kernel density estimation [Silverman 1986] is used. That leads to:

$$L(x, \omega_0) = \frac{1}{h^2} \sum_{p=1}^N \Delta\Phi_p f_r(x, \omega_p, \omega_0) K\left(\frac{x - x_p}{h}\right) \quad (3)$$

where N is the number of stored photons, $\Delta\Phi_p$ the power of the photon p , $f_r(x, \omega_p, \omega_0)$ the bidirectional reflectance distribution function and K the kernel function of bandwidth h (i.e. scaling factor).

The interest of this kernel density estimation is simple: the radiance is a mean of photon power weighted by the kernel function centered in the estimation point (direct approach). It can also be seen as the sum of the N kernel functions centered on the N photons weighted by their power (dual approach). Figure 3 shows the direct and dual approaches of density estimation.

Photon Splatting uses a dual approach of density estimation. In the direct approach used in Photon Mapping [Jensen 2001], the estimation is done using the nearest neighbour method to estimate the

value for each evaluation point. On the contrary, in the dual approach, the contribution of each sample is added to all evaluation points it influences. The idea of Photon Splatting is that a photon can be seen as a little disc, and all the evaluation points influenced by this photon are the points covered by the projection of the photon on the surface where radiance is evaluated. Then, the disc is the support of estimation and the value of each influenced point is given by the energy of the photon weighted by the kernel function. Photon Splatting is faster than Photon Mapping because photons are treated sequentially, then we do not need to search neighbour photons, which is costly, nor to sort the photon map.

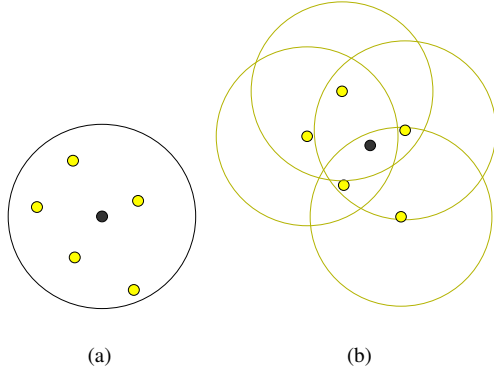


Figure 3: Density estimation approaches. (a) Direct: the radiance at estimation point (black) is the sum of the photon power weighted by the kernel function centered on the estimation point. (b) Dual: the radiance at estimation point is the sum of the photon power weighted by the kernel function centered on the photon.

3.3 Discussion of Photon Mapping for participating media

The Photon Mapping method [Jensen 2001] is a two pass method. Firstly, light is propagated through the medium using standard ray tracing techniques. A virtual particle, called photon by analogy with the corpuscular theory of the light, is used to store energy at the interaction points of the light with the medium. The data structure obtained is called a photon map. The second pass consists of reconstructing the radiance seen by the observer by throwing a ray from the eye through the media. Then the radiance gathered by the ray is computed using a numerical integration scheme, i.e. cutting the ray in little segments for which radiance is evaluated by a recursive expression of the volume transfer equation. This method is called recursive Ray Marching.

The most important limitation of the method comes from the evaluation of in-scattering. Actually, this comes from the nearest neighbour method. For each segment, an evaluation point is chosen. Then the nearest photons to this point have to be found in the photon map. Considering a photon map, implemented using a balanced kd-tree [Bentley 1979], containing N photons, the average complexity of searching the k nearest photons is $O(k + \log N)$. As Photon Mapping is a stochastic method it converges when the number of samples tends to infinity, i.e. the accuracy of the simulation rises with k and N . The problem here is that we have to search in the photon map once for each step of the ray and the number of steps can have a high dependency on the variations of the medium. For a ray discretised in n steps, the complexity is then $O(n(k + \log N))$. Furthermore, two evaluations done for two neighbour segments could use the same photons which make us

perform redundant searches of these photons in the photon map.

4 Fast participating media rendering

In this section, we present our method. Firstly, section 4.1 shows the iterative formulation of the volume transfer equation we use. This expression permits us to treat separately the less expensive simulation of emission, absorption and out-scattering (4.2) and the very expensive simulation of in-scattering (section 4.3). Finally, we present our results in section 4.4.

4.1 Iterative volume transfer equation

In order to minimize the computation of the previously discussed method (section 3.3), we reformulate the problem in a better suited expression. Equation 1 can be solved using numerical integration (figure 4). This integration is done by cutting the ray in little segments $[x_i, x_{i+1}]$ on which all the coefficients are considered constant. Photon Mapping uses a recursive expression of the radiative transfer equation. We choose a distinct approach, and then we reformulate equation 1 in an iterative form.

This equation provides distinct terms that can be computed separately. For a ray divided in n little segments, we have:

$$L(x_n) = \prod_{i=0}^{n-1} e^{-\tau(x_i, x_{i+1})} L(x_0) + \sum_{j=1}^n P(j) \int_{x_{j-1}}^{x_j} e^{-\tau(u, x_j)} \sigma_a(u) L_e(u) du + \sum_{j=1}^n P(j) \int_{x_{j-1}}^{x_j} e^{-\tau(u, x_j)} \frac{\sigma_s(u)}{4\pi} L_\omega(u) du \quad (4)$$

with

$$P(j) = \begin{cases} 1 & \text{if } j = n, \\ \prod_{i=j}^{n-1} e^{-\tau(x_i, x_{i+1})} & \text{else.} \end{cases}$$

and where $L(x_0)$ the radiance behind the media. Equation 4 gives a separate formulation of the contribution of each light phenomena which occurs in the media. Then we can use distinct methods to compute each term. Section 4.2 explains our method to treat emission, absorption and out-scattering terms and section 4.3 presents our method to compute in-scattering.

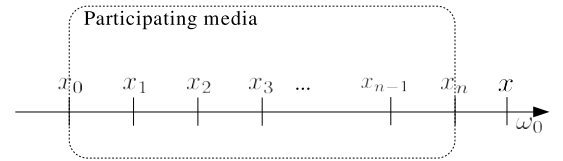


Figure 4: Geometry associated with Ray Marching.

4.2 Emission, absorption and out-scattering simulation

Let us consider emission, absorption and out-scattering in equation 4. We note $L_{RM}(x_n)$ the radiance due to these phenomena. The cutting of the ray is done so that on each segment of the ray the properties of the medium are constant and that the optical thickness is supposed to be constant and near 0 on each segment (i.e. adaptive ray marching). In this case, $e^{-\tau(u, x)} \approx 1$. This means that radiance is computed assuming that it is constant over the segment.

With these simplifications, we can express $L_{RM}(x_n)$ as a discrete equation:

$$L_{RM}(x_n) = \prod_{i=0}^{n-1} e^{-\sigma_t(x_i)\Delta x_i} L(x_0) + \sum_{j=1}^n P(j)\sigma_a(x_j)L_e(x_j)\Delta x_i \quad (5)$$

and

$$P(j) = \begin{cases} 1 & \text{if } j = n, \\ \prod_{i=j}^{n-1} e^{-\sigma_t(x_i)\Delta x_i} & \text{else.} \end{cases}$$

where $\Delta x_i = \|x_i, x_{i+1}\|$ the length of each segment.

This equation is evaluated using a ray marching like method. The computed radiance is stored for each ray in order to combine it with the in-scattering radiance.

4.3 Photon Splatting for in-scattering simulation

In this section, we describe our main contribution. Our method first propagates the light flux through the medium and store the energy in a volume photon map. It doesn't take into account direct illumination explicitly. Actually, we want to handle as many extended light sources as we want. And tracing enough shadow rays to extended sources (as in Photon Mapping) in order to reduce noise is more costly than simply tracing more photons. Then we choose to deal with all illumination at the same time by direct estimation of the photon map. Hence, we store all the interactions between light and medium in the photon map even the first ones.

We chose to use the photon map storage in order to be view independent. Nevertheless, one can choose to splat the photons on the fly, in order to not be limited on the number of photons and hence on the simulation accuracy.

Our method differs from Photon Mapping in the second pass. Reformulating the transfer equation as an iterative equation permits us to treat emission, absorption and out-scattering as described in section 4.2. We can also compute separately the in-scattering contribution which is very interesting.

We note $L_{IS}(x_n)$ the radiance due to in-scattering. We have:

$$L_{IS}(x_n) = \sum_{j=1}^n P(j) \int_{x_{j-1}}^{x_j} e^{-\tau(u, x_j)} \frac{\sigma_s(u)}{4\pi} L_\omega(u) du \quad (6)$$

Using the same approximations as for equation 5, we obtain:

$$L_{IS}(x_n) = \sum_{j=1}^n P(j) \frac{\sigma_s(x_j)}{4\pi} \Delta x_j L_\omega(x_j) \quad (7)$$

We choose to compute $L_\omega(x_j)$ using density estimation of the photon map. Considering the relation between flux and radiance:

$$L(x, \omega) = \frac{d^2 \Phi(x, \omega)}{\sigma_s(x) d\omega dV} \quad (8)$$

where dV is a volume element, we can now integrate equation 2 using the previous relation and density estimation. We obtain:

$$L_\omega(x_j) = \sum_{p=1}^N \left[K_h(x_j - x_p) p(x_j, \omega_0, \omega_p) \frac{\Delta \phi(x_p, \omega_p)}{\sigma_s(x_j) \Delta V} \right] \quad (9)$$

with N the total number of photons, K_h the kernel function, x_p the photon position, ω_p the photon incident direction and ΔV is the small volume containing the photons.

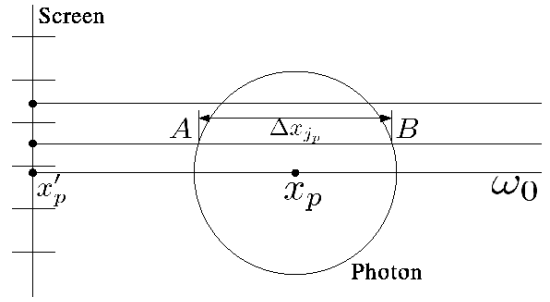


Figure 5: Geometry associated with Photon Splatting.

Replacing L_ω in equation 7, we have:

$$L_{IS}(x_n) = \sum_{j=1}^n P(j) \frac{\sigma_s(x_j)}{4\pi} \Delta x_j \times \sum_{p=1}^N K_h(x_j - x_p) p(x_j, \omega_0, \omega_p) \frac{\Delta \phi(x_p, \omega_p)}{\sigma_s(x_j) \Delta V} \quad (10)$$

We can switch the sums of equation 10:

$$L_{IS}(x_n) = \sum_{p=1}^N \sum_{j=1}^n \left[P(j) \frac{1}{4\pi} \Delta x_j K_h(x_j - x_p) \times p(x_j, \omega_0, \omega_p) \frac{\Delta \phi(x_p, \omega_p)}{\Delta V} \right] \quad (11)$$

However, in equation 11, we notice that for each photon p , there will be only a few terms of the sum on j that have a non null value. Actually, the kernel function is equal to 0 for all the segments that are further than the kernel bandwidth. Then we can replace the sum on j by a single non null term of index j_p . Finally, we have:

$$L_{IS}(x_n) = \frac{1}{4\pi} \sum_{p=1}^N \left[P(j_p) p(x_{j_p}, \omega_0, \omega_p) \times \frac{K_h(x_{j_p} - x_p) \Delta x_{j_p} \Delta \phi(x_p, \omega_p)}{\Delta V} \right] \quad (12)$$

Equation 12 gives the radiance for a ray corresponding to a pixel of the screen. As we want the estimate to be done in the screen space, we don't have information on x_{j_p} . But, we can see that the term Δx_{j_p} corresponds to the portion of the ray influenced by the photon (figure 4.3). Furthermore, the kernel function gives the way the photon influences the ray. Then we integrate the kernel function on the path Δx_p of the ray through the photon in order to know the contribution of the photon for a ray. For the other terms that need x_{j_p} we pose that $x_{j_p} \approx x_p$.

This leads to our algorithm. Our idea is to project the photon as if it was a little sphere. Projecting a sphere is similar to projecting a billboard disc. That way, the computation can be done for all the pixels that are influenced by the photon using a texture. This texture is applied to the photon billboard disc centered at the photon position. This precomputed texture takes into account the 3D kernel function and the path of a ray in the photon i.e. each pixel (u, v) of the texture is equal to :

$$T(u, v) = \frac{1}{\frac{4}{3}\pi r^3} \int_A^B K(x_{u,v}(t) - x_p) dt \quad (13)$$

Where r is the photon virtual radius and A and B are defined as in figure 4.3. Then for each photon, we only have to compute the attenuation $P(j_p)$ for the photon. This is done using a ray marching toward the photon position. Here we make the approximation that the attenuation for each pixel covered by the photon projection is equal to the attenuation toward the photon position. This approximation introduces bias in the solution. Nevertheless, this bias is proportional to the kernel bandwidth, and the simulation converges when the bandwidth tends towards 0. As we throw one ray per photon to compute attenuation, we make redundant computations which Jensen's method would not do. But, as it is illustrated in the results section 4.4, our method is still faster than Photon Mapping. Furthermore, we are thinking about projecting photons from front to back in order to construct the attenuation iteratively and then save more computing time. After computing the photon attenuation, the textured disc is projected on the screen. The texture is modulated by the photon energy, the phase function weight and the attenuation. This is done for each photon and radiance is accumulated in all pixels of the screen.

Once we have computed on one hand the emission, attenuation and out-scattering image and on the other hand the in-scattering radiance image, the last thing is to sum the two images. Algorithm 1 sums up our method.

```

Select visible photons
for each visible photon with power  $\Phi_p$  do
  Compute photon weight  $w$  using ray marching
  Project the photon on the screen
  for each pixel  $p$  covered by the photon projection do
    Get the filter weight  $w_f$  from filter texture
     $L(p) += w * w_f * \Phi_p$ 
  end for
end for
Compute emission, absorption and out-scattering image using ray marching
Sum the previous image with the in-scattering image

```

Algorithm 1: Photon Splatting algorithm.

4.4 Results

We have tested our algorithm on a wide variety of participating media. We wanted to show that our method offers the same capacities as Photon Mapping for equivalent quality with better performances.

Capacities Figure 6 illustrates the handle of both volume shadows and light caustics. In our method, we did not use a special photon map in order to stock caustic photons. This implies that our volume caustics look smooth. Actually, we use the same kernel bandwidth for all photons. But our algorithm could easily use a special photon map for caustic photons. Nevertheless, we are working on new adaptive strategies for choosing the photon size according to the media properties and photon density. And we think about including in these strategies a way to treat with caustics without handling explicitly caustic photons. In the same way, shadow boundaries should be more tidy, and shadow zones are suffering of noise because of the low photon density in these regions.

Quality We compared the image quality obtained using Photon Mapping and our algorithm. Figure 7 show a visual comparison of the rendering of an anisotropic cloud with both methods. In table 1, we measure the error between the previous images. It appears that we achieve the same image quality as Photon Mapping.

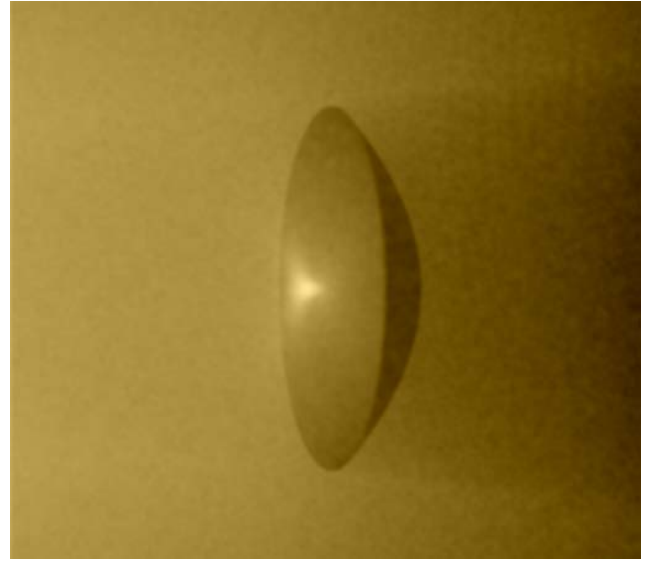


Figure 6: Rendering of a parabola in an homogeneous cloud lighted by a distant extended light. The parabola generates a caustic and volume shadows.

	Average radiance	Standard deviation	Rendering time (s)
Photon Mapping	29.55	34.37	99096
Photon Splatting	30.82	35.96	1497

Table 1: Pixel average radiance and standard deviation comparison for the images of figure 7.

Efficiency We have tested our implementation using a 2.4GHz Pentium 4 system. Tables 2,3 and 4 gives the comparison of the computing times obtained with Photon Mapping and our method according to the evolution of the image size, the photon map size and the kernell bandwidth (i.e. the search distance for Photon Mapping and the photon size in our method). The results in these tables are given for the rendering of the same cloud as in figure 7. According to the obtained results, we see that our algorithm is ten times faster than Photon Mapping in the worst case. However, rendering times are still long. Actually, our software implementation is dependent on the image resolution and the size of the photon. As a matter of fact, all the pixels covered by the photon projection have to be treated one after the other in order to compute the contribution of the photon to the pixel (access the weight texture and the photon energy, compute the phase function weight and do the product). Therefore, computing times lengthen with the resolution of the image and the kernell bandwidth as more pixels are covered by the photon projection. Of course, computing costs also depend linearly on the number of photons in the photon map, as the computing of the contribution has to be done for each photon. But, in common application cases, our method is always faster than Photon Mapping.

5 GPU optimization

With the last generation of graphics processors, it is possible to do floating point rendering using floating point textures. Then, our idea is to use the optimized projection and texturing of graphics processors to drastically reduce the computing times of our method. As a matter of fact, these points are the part of the algorithm which



(a)



(b)

Figure 7: Rendering of an anisotropic cloud using four million photons. 7(a) : Photon Mapping. 7(b) : Our algorithm.

take the most time.

5.1 Principle

We take advantage of the parallelism between the CPU and GPU and only port on the GPU the computing it is suited to. We render the photons using a floating point offscreen pBuffer. The weight texture of the photons is precomputed and stored in a 16-bits floating point texture. The phase function weight and the attenuation of the photon are computed using the CPU. The product of these terms with the photon energy is done and the obtained value is passed to the graphic processor using texture coordinates. Then the photon

Image size	256 × 256	512 × 512	1024 × 1024	2048 × 2048
Photon Mapping	912	4431	16005	63514
Photon Splatting	64	196	741	3463
Speed up ratio	14	22	21	18

Table 2: Comparison of rendering times (in seconds) obtained using Photon Mapping and Photon Splatting according to the image size. These results are given for a photon map of one million photons and a photon size (i.e. search distance for Photon Mapping) of 10.0 meters. The rendered cloud is the same as in figure 7.

Number of photons	1M	2M	3M	7M
Photon Mapping	4431	7424	9629	14781
Photon Splatting	196	381	514	1191
Speed up ratio	22	19	18	12

Table 3: Comparison of rendering times (in seconds) obtained using Photon Mapping and Photon Splatting according to the photon map size. These results are given for an image of 512×512 pixels and a photon size (i.e. search distance for Photon Mapping) of 10.0 meters. The rendered cloud is the same as in figure 7.

Photon size	1.0	5.0	10.0	20.0	30.0
Photon Mapping	749	1322	4431	13649	21913
Photon Splatting	32	67	196	708	1563
Speed up ratio	23	19	22	19	14

Table 4: Comparison of rendering times (in seconds) obtained using Photon Mapping and Photon Splatting according to the photon size (i.e. search distance for Photon Mapping). These results are given for an image of 512×512 pixels and a photon map of one million photons. The rendered cloud is the same as in figure 7.

is rendered (as a quad textured with the weight texture) using a fragment program that combines the photon energy passed through texture coordinates and the kernel texture. The radiance is accumulated in the rendering buffer using 16-bits floating point blending. Finally the computed image is read from the pBuffer and combined with the absorption, emission and out-scattering image using the CPU.

5.2 Results

Tables 5(a), 5(b) and 5(c) show the results obtained using GPU optimization for the same tests as presented in tables 2, 3 and 4. All the tests were done using the same processor as for the software version and a NVIDIA Geforce 6800 GT.

The computing times using this GPU optimization are really interesting. Actually, computing times have linear complexity on the image resolution and the photon size but the linear coefficient is very small.

However some problems remain. As a matter of fact, current graphics processors only permit 16-bits floating point blending

Image size	256 × 256	512 × 512	1024 × 1024	2048 × 2048
Rendering time (s)	23.34	23.79	34.71	39.59
Speed up ratio	39	186	461	1601

(a)

Number of photons	1M	2M	3M	7M
Rendering time (s)	23.79	48.57	72.92	167.71
Speed up ratio	186	152	132	88

(b)

Photon size	1.0	5.0	10.0	20.0	30.0
Rendering time (s)	23.92	24.09	23.79	23.79	27.01
Speed up ratio	31	54	186	573	811

(c)

Table 5: Evolution of rendering times using GPU optimization according to the image size (5(a)), the number of photons (5(b)) and the photon size (i.e. search distance in Photon Mapping) (5(c)). These results were obtained in the same conditions as for tables 2, 3 and 4. The speed up ratio is given in front of the Photon Mapping rendering times.

which leads to accuracy problems. Actually, we have to project and blend all the photons on the screen. In 16-bits format, accuracy is lost when adding high numbers with little ones. However, we have to blend very small photon energies a lot of times per pixel, then after a certain number of additions (i.e. a thousand photons per pixel) the pixel value becomes high compared to the photon weight causing the loss of accuracy. Nevertheless, this loss of accuracy is only a problem when images are used to do measurements of a physical simulation, but it isn't if we want to do pseudo-realistic rendering like for communication or entertainment purpose.

This problem will be solved naturally when graphic processors can do 32-bits floating point blending. Until then, we have solved the problem using a multipass method. In order to avoid the loss of accuracy, we only treat a number of photons that make us sure not to have a loss of accuracy per pass and add the radiance from the frame buffer to a float buffer using the CPU. This causes a loss of performance due to implementation but we think that our method is still interesting.

6 Conclusion and future work

We have presented a method that considerably reduces the rendering times of participating media. We used an iterative expression of the volume transfer equation in order to extract the computing of in-scattering from the Ray Marching. Thus, we can compute separately emission, absorption, and out-scattering on one hand and in-scattering on the other hand. Our method optimizes rendering times

specially for in-scattering which is the costly part of the simulation using a dual approach of kernel density estimation. Our method is faster than Photon Mapping for equal quality.

We use kernel density estimation with a constant bandwidth which can result in smoothing some details. We are thinking about using adaptive density estimation in order to adapt the filter size according to media properties and photon density. This would be suited to render accurate participating media boundaries and caustics, and to make the clouds less smooth, and hence more realistic. We did not discuss the choice of the kernel. In fact, we use the optimal Epanechnikov kernel for all our renderings. But we want to try other kernels such as the ones used for EWA volume splatting [M. Zwicker and Gross 2001; Wei Chen and Pfister 2004]

Finally, we compute photon attenuation for each photon using a raytracing technique. We think about projecting photons in a front to back scheme in order to avoid redundant calculations of attenuation.

References

- BENTLEY, J. L. 1979. Multidimensional binary search trees in database applications. *IEEE Trans. on Soft. Eng.* 5, 4 (July), 333–340.
- BLASI, P., LESAC, B., AND SCHLICK, C. 1993. A rendering algorithm for discrete volume density objects. *Computer Graphics Forum* 12, 3 (Sept.), 201–210.
- BLASI, P., LESAC, B., AND SCHLICK, C. 1994. An importance driven monte-carlo solution to the global illumination problem. *5th EG Workshop on Rendering* (June), 173–183.
- CHANDRASEKAR, S. 1950. *Radiative Transfer*. Oxford Univ. Press.
- HARRIS, M. J., AND LASTRA, A. 2001. Real-time cloud rendering.
- JENSEN, H. W. 2001. *Realistic Image Synthesis Using Photon Mapping*. A K PETERS LTD edition.
- LAFORTUNE, E., AND WILLEMS, Y. 1996. Rendering participating media with bidirectional path tracing. *6th EG Workshop on Rendering* (June), 92–101.
- LAVIGNOTTE, F., AND PAULIN, M. 2003. Scalable Photon Splatting for Global Illumination. In *Graphite 2003 (International Conference on Computer Graphics and Interactive Techniques in Australasia and South East Asia)*, Melbourne, Australia., ACM SIGGRAPH, 1–11.
- LSZL, S.-K. 1999. Monte-carlo global illumination methods - state of the art and new developments. *SCCG'99, Invited talk*.
- M. ZWICKER, H. PFISTER, J. V. B., AND GROSS, M. 2001. Ewa volume splatting. In *IEEE Visualization 2001*, 29–36.
- MAX, N. 1994. Efficient light propagation for multiple anisotropic volume scattering. *5th EG Workshop on Rendering* (June), 57–104.
- NISHITA, T., DOBASHI, Y., AND NAKAMAE, E. 1996. Display of clouds taking into account multiple anisotropic scattering and sky light. *Computer Graphics, Proc. of ACM SIGGRAPH 96*, 379–386.
- PATTANAIK, S., AND MUDUR, S. 1993. Computation of global illumination in a participating medium by monte carlo simulation. *The Journal of Vis. and Comp. Animation* 4, 3, 133–152.

- PEREZ, F., PUEYO, X., AND SILLION, F. X. 1997. Global illumination techniques for the simulation of participating media. In *Rendering Techniques '97 (Proceedings of the Eighth Eurographics Workshop on Rendering)*.
- RUSHMEIER, H., AND TORRANCE, K. 1987. The zonal method for calculating light intensities in the presence of a participating medium. *Computer Graphics, Proc. of ACM SIGGRAPH 87* 21, 4, 293–302.
- SIEGEL, R., AND HOWELL, J. R. 1992. *Thermal Radiation Heat Transfer*. 3rd edition, Hemisphere Publishing.
- SILVERMAN, B. W. 1986. Density estimation for statistics and data analysis. *Chapman and Hall, London and New York*.
- STUERZLINGER, W., AND BASTOS, R. 1997. Interactive Rendering of Globally Illuminated Glossy Scenes. In *Rendering Techniques '97 (Proceedings of the 8th Eurographics Workshop on Rendering)*, 93–102.
- VEACH, E., AND GUIBAS, L. J. 1997. Metropolis light transport. *SIGGRAPH 97 Proceedings* (Aug.), 65–76.
- WEI CHEN, LIU REN, M. Z., AND PFISTER, H. 2004. Hardware-accelerated adaptive ewa volume splatting. In *Proceedings of IEEE Visualization 2004*, 67–74.

Photon Splatting For Participating Media

Boudet, Pitot, Prاتمarty and Paulin

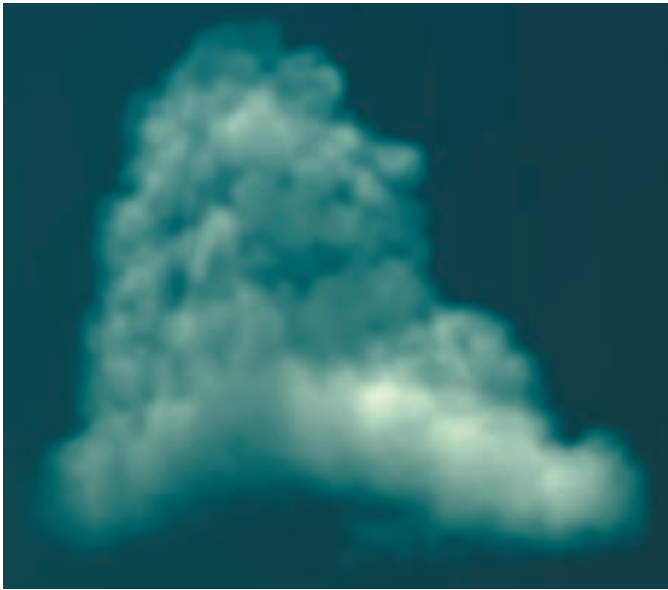


Figure 1: Rendering of an anisotropic cloud using four million photons with our algorithm.

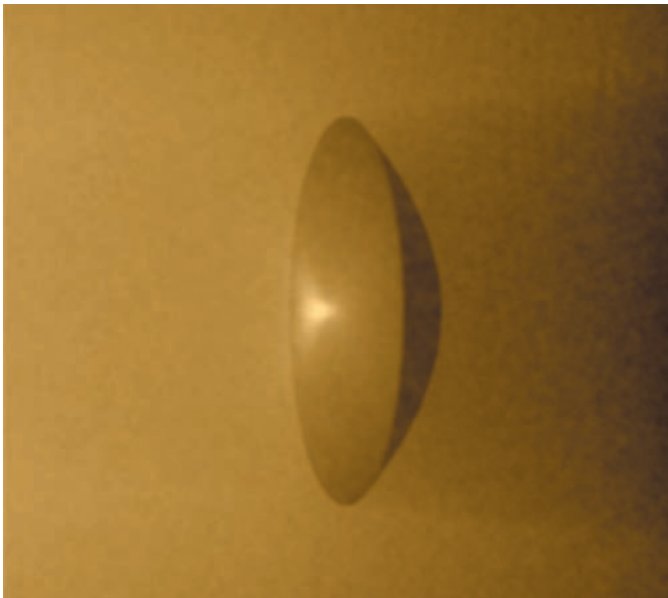


Figure 2: Rendering of a parabola in an homogeneous cloud lighted by a distant extended light. The parabola generates a caustic and volume shadows.

A Novel Color Region Homogenization and Its Application in Improving Skin Detection Accuracy

Akbari and Nakajima



Figure 2: Top: Original image, Middle: Skin classifier result, Bottom: Result after region growing



Figure 3: Result of region growing with spatial influence coefficient